TP1 : cadrer une application IA et préparer l'environnement **Cours :** Développement et déploiement d'applications intelligentes · **Semaine 1** · **Durée :** 2 h · **Travail :** en binôme (membre A et membre B) ## Objectifs À la fin de ce TP, chaque binôme dispose : 1. de trois fiches de cadrage argumentées, une par mini-cas ; 2. d'un dépôt GitHub privé `sentiment-app`, structuré, où les deux membres ont poussé au moins un commit ; 3. d'un environnement Python reproductible, vérifié par `python scripts/check_setup.py` et par `pytest`. Ce dépôt est le point de départ des dix semaines suivantes : rien de ce que vous faites aujourd'hui ne sera jeté. ## Ce que contient le kit | Fichier | Rôle | | --- | --- | | `README.md` | Modèle de README du dépôt : installation, structure, tableau du binôme | | `TESTS.md` | Démarche à suivre pour le test, résultats attendus à chaque étape | | `requirements.txt` | Dépendances de tout le semestre, versions figées | | `.gitignore`, `.env.example`, `pytest.ini` | Fichiers de configuration à garder tels quels | | `docs/cadrage/TEMPLATE.md` | Modèle de fiche de cadrage | | `docs/cadrage/cas1-pharmacie.md`, `cas2-reglement.md`, `cas3-avis-negatifs.md` | Les trois fiches à remplir (partie A) | | `docs/cadrage/EXEMPLE-corrige.md` | Un exemple rempli, pour calibrer le niveau de détail | | `docs/cadrage/BANQUE-DE-CAS.md` | Douze cas de rechange (entreprise, citoyens, université, État) | | `scripts/setup.sh`, `scripts/setup.ps1` | Mise en place du poste (Linux et macOS, Windows) | | `scripts/check_setup.py` | Diagnostic lisible de l'environnement | | `scripts/new_cadrage.py` | Crée une fiche supplémentaire à partir du modèle | | `tests/` | 38 tests : environnement (23), fiches de cadrage (12), historique Git (3) | | `.github/workflows/ci.yml` | Intégration continue : `pytest` à chaque push | | `data/sample_reviews.csv` | Dix avis d'exemple, utiles dès la semaine 2 | ## Partie A : cadrage (50 min, en binôme) ### A1. Lire l'exemple corrigé (5 min) Ouvrir `docs/cadrage/EXEMPLE-corrige.md`. Chaque rubrique tient en une à trois phrases, et la justification cite les critères de la grille par leur lettre : - **(a)** données étiquetées ; - **(b)** vérifiabilité ; - **(c)** coût par requête ; - **(d)** conséquence d'une erreur. ### A2. Remplir les trois fiches (40 min) | Fiche | Cas | Indices à discuter | | --- | --- | --- | | `cas1-pharmacie.md` | Une pharmacie veut détecter les ordonnances incomplètes | Les champs obligatoires sont-ils connus ? Que coûte une ordonnance validée à tort ? Données de santé | | `cas2-reglement.md` | Une école veut répondre aux questions des étudiants sur le règlement intérieur | Les documents existent déjà. La réponse doit-elle être vérifiable ? A-t-on des questions étiquetées ? | | `cas3-avis-negatifs.md` | Un site e-commerce veut prioriser les avis clients négatifs | Des milliers d'avis. Peut-on les étiqueter ? Que coûte un avis mal classé ? | Chaque fiche comporte huit rubriques obligatoires : besoin en une phrase, utilisateur final, approche retenue, justification, données nécessaires et leur origine, métrique de succès et seuil d'acceptation, conséquence d'une erreur et validation humaine prévue, risques éthiques ou de confidentialité. Une neuvième rubrique est facultative : approche écartée et pourquoi. Règles : - une seule case cochée (`- [x]`) dans « Approche retenue » ; - la justification cite au moins deux critères par leur lettre, par exemple « (a) ... (d) ... » ; - chaque rubrique obligatoire contient au moins une phrase complète ; - deux approches peuvent se défendre pour un même cas : c'est la justification qui est notée, pas la case cochée. Méthode : un membre tient le clavier, l'autre parcourt l'arbre de choix dans l'ordre du cours et conteste. 1. Logique connue et stable ? Règles métier. 2. Sinon, données étiquetées ? Machine learning. 3. Sinon, réponse tirée de documents ? RAG. 4. Sinon : modèle génératif seul. Inversez les rôles à chaque fiche. Pour aller plus loin (facultatif) : choisir un cas dans `docs/cadrage/BANQUE-DE-CAS.md` et le cadrer en quatrième fiche avec `python scripts/new_cadrage.py cas4- "Titre du cas"`. ### A3. Vérifier (5 min) ```bash pytest tests/test_cadrage.py -q # attendu : 12 passed ``` Tant qu'une rubrique est vide, qu'aucune ou plusieurs approches sont cochées, ou que la justification cite moins de deux lettres, le test correspondant échoue et nomme la fiche fautive. ## Partie B : environnement et dépôt (70 min) ### B1. Outils système (10 min, chacun) ```bash python --version # 3.11.x attendu, 3.10 toléré git --version code --version # VS Code, facultatif mais recommandé ``` Windows : installer Python depuis python.org en cochant « Add python.exe to PATH » ; utiliser `py -3.11` si plusieurs versions cohabitent. Éviter Python 3.12 et 3.13 : certaines versions figées du semestre ne s'y installent pas. ### B2. Créer le dépôt GitHub (10 min, membre A) 1. Dans l'organisation GitHub du cours : « New repository », nom `sentiment-app`, visibilité **Private**, sans README, sans `.gitignore`, sans licence (le kit fournit tout cela). 2. Settings › Collaborators : le membre B en **Write**, l'enseignant en **Read**. 3. Copier l'URL HTTPS de clonage. ### B3. Initialiser le dépôt à partir du kit (15 min, membre A) ```bash cd sentiment-app

git init git add -A git commit -m "Semaine 1 : structure du depot et kit TP1" git branch -M main git remote add origin https://github.com//sentiment-app.git git push -u origin main ``` Si Git refuse le commit faute d'identité : ```bash git config --global user.name "Prénom Nom" git config --global user.email "prenom.nom@ecole.ma" ``` Chaque membre utilise son propre nom : c'est ce nom que `git shortlog` compte en B6. Vérifier sur GitHub que `.venv/` n'apparaît pas et que `docs/cadrage/` contient les fiches. Un premier passage rouge de la CI est normal à ce stade (fiches vides, un seul auteur) : il passe au vert en B6. ### B4. Installer l'environnement (15 min, chacun) Le membre B clone, puis chacun lance le script de mise en place ; le membre A le lance dans son dossier existant. ```bash git clone https://github.com//sentiment-app.git cd sentiment-app bash scripts/setup.sh # Linux, macOS, Git Bash powershell -ExecutionPolicy Bypass -File scripts\setup.ps1 # Windows ``` Le script vérifie Python, crée `.venv`, installe `requirements.txt`, écrit `requirements.lock`, puis lance le diagnostic. Comptez 5 à 10 minutes la première fois. Résultat attendu : `0 erreur(s)` au diagnostic. `requirements.lock` est régénéré sur chaque poste : un seul membre le commite. ### B5. Un commit par membre (10 min, chacun) ```bash git pull git add README.md git commit -m "README : ajout de au tableau du binome" git push ``` En cas de refus (`rejected, non-fast-forward`) : `git pull`, puis `git push`. En cas de conflit sur `README.md` : garder les deux lignes du tableau, supprimer les marqueurs `<<<<<<<`, `=======`, `>>>>>>>`, puis `git add`, `git commit`, `git push`. ### B6. Vérification finale (10 min, à deux) ```bash git pull python scripts/check_setup.py # attendu : 0 erreur(s), 0 avertissement(s) pytest -q # attendu : 38 passed ``` Pousser une dernière fois et vérifier que la CI est verte dans l'onglet Actions. La démarche de test complète, avec les résultats attendus à chaque étape, est dans `TESTS.md`. ## Déroulé minuté | Étape | Durée | Qui | Résultat attendu | | --- | --- | --- | --- | | A1 Lire l'exemple corrigé | 5 min | le binôme | niveau de détail compris | | A2 Remplir les trois fiches | 40 min | le binôme | trois fiches, une case cochée, deux critères cités | | A3 Vérifier les fiches | 5 min | le binôme | `pytest tests/test_cadrage.py` vert | | B1 Outils système | 10 min | chacun | Python 3.11, Git, VS Code présents | | B2 Créer le dépôt GitHub | 10 min | membre A | dépôt privé `sentiment-app`, membre B collaborateur | | B3 Initialiser depuis le kit | 15 min | membre A | premier commit poussé sur `main` | | B4 Installer l'environnement | 15 min | chacun | `.venv` créé, `0 erreur(s)` au diagnostic | | B5 Un commit par membre | 10 min | chacun | `git shortlog -sn` liste les deux membres | | B6 Vérification finale | 10 min | le binôme | `pytest -q` vert, CI verte | ## Repères de test | État | `check_setup.py` | `pytest` | | --- | --- | --- | | Kit décompressé, pas de dépôt Git, fiches vides | 1 erreur, 3 avertissements | 3 failed, 31 passed, 4 skipped | | Dépôt Git initialisé, un seul auteur, fiches vides | 0 erreur, 4 avertissements | 4 failed, 34 passed | | Fin du TP : fiches remplies, deux auteurs | 0 erreur, 0 avertissement | 38 passed | ## Livrables (sur la branche `main`, à la fin de la séance) - [ ] `docs/cadrage/cas1-pharmacie.md`, `cas2-reglement.md`, `cas3-avis-negatifs.md` remplies - [ ] `README.md` avec le tableau du binôme complété - [ ] `requirements.lock` généré - [ ] Structure de dossiers conforme : `data/`, `models/`, `app/`, `ui/`, `tests/`, `scripts/`, `docs/cadrage/` - [ ] Un commit par membre visible dans `git shortlog -sn` - [ ] CI verte sur GitHub ## Grille de notation (TP1 sur 10) | Critère | Points | Vérification | | --- | --- | --- | | Fiches de cadrage : approche justifiée par deux critères, rubriques remplies | 4 | Lecture + `pytest tests/test_cadrage.py` | | Dépôt clonable, `pip install -r requirements.txt` sans erreur | 2 | Clone frais par l'enseignant | | Structure et `.gitignore` conformes, `.venv/` absent de l'historique | 2 | `pytest tests/test_environment.py` | | Un commit par membre, messages de commit explicites | 2 | `git shortlog -sn`, `git log --oneline` | La note se joue sur un clone frais : avant de partir, clonez votre dépôt dans un autre dossier, installez, puis lancez `check_setup.py` et `pytest -q` (étape 6 de `TESTS.md`). ## Pièges fréquents | Symptôme | Cause | Correction | | --- | --- | --- | | Le dépôt pèse 300 Mo | `.venv/` commité | `git rm -r --cached .venv`, vérifier `.gitignore`, commit | | `ModuleNotFoundError: pandas` | venv non activé | `source .venv/bin/activate`, puis `which python` dans `.venv` | | `test_virtualenv_active` échoue | pytest lancé hors du venv | activer le venv, ou `.venv/bin/python -m pytest` | | `2 attendus (un par membre du binôme)` | un seul membre a poussé, ou même `user.name` sur les deux postes | chaque membre fait un vrai commit avec sa propre identité (B5) | | `la justification doit citer au moins deux critères` | justification sans `(a)`, `(b)`... | citer les lettres explicitement | | `Activate.ps1 cannot be loaded` | politique d'exécution PowerShell | lancer avec `-ExecutionPolicy Bypass` | | `Support for password authentication was removed` | GitHub refuse le mot de passe du compte | jeton d'accès personnel ou `gh auth login` | | `pip install` très

long | sentence-transformers et mlflow sont lourds | `pip install pandas scikit-learn pytest` d'abord, le reste en fin de séance |